

NS-BRKGA para o Multi-Objective Time- and Machine-Dependent Scheduling Problem

Luis H. P. Mendes Fábio L. Usberti Mário C. San Felice Carlos E. de Andrade

Instituto de Computação — Universidade Estadual de Campinas (UNICAMP)
Departamento de Computação — Universidade Federal de São Carlos (UFSCar)
AT&T Labs Research

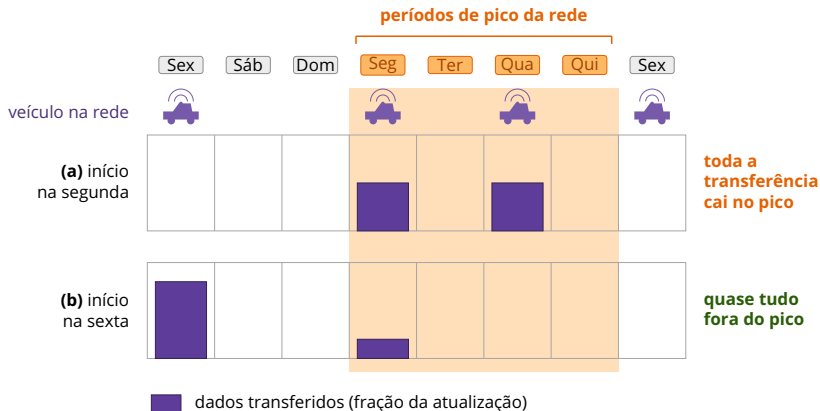
`luis.mendes@ic.unicamp.br`

LVIII Simpósio Brasileiro de Pesquisa Operacional — SBPO 2026

Em FOTA, o que se decide é *quando* começar

Um veículo conectado:

- fica na rede **poucas horas por dia;**
- sob **setores diferentes** a cada dia;
- iniciada a transferência, a rede **não a interrompe.**



Cenário de FOTA para carros conectados — Andrade et al. (2019)

Do FOTA ao TMDSP

O FOTA é modelado pelo *Time- and Machine-Dependent Scheduling Problem* (TMDSP), de Andrade et al. (2019):

O FOTA vira escalonamento

- J — **tarefas**: os veículos a atualizar.
- M — **máquinas**: os setores da rede.
- T — **períodos**: os intervalos do horizonte.

Regras de um escalonamento viável

Do FOTA ao TMDSP

O FOTA é modelado pelo *Time- and Machine-Dependent Scheduling Problem* (TMDSP), de Andrade et al. (2019):

O FOTA vira escalonamento

- J — **tarefas**: os veículos a atualizar.
- M — **máquinas**: os setores da rede.
- T — **períodos**: os intervalos do horizonte.

Regras de um escalonamento viável

- Cada tarefa usa **uma máquina por período**, no máximo;
- é **preemptável entre períodos** e fica *ativa* até atender toda a sua demanda;
- C_j é o **último período ativo** de j .

Do FOTA ao TMDSP

O FOTA é modelado pelo *Time- and Machine-Dependent Scheduling Problem* (TMDSP), de Andrade et al. (2019):

O FOTA vira escalonamento

- J — **tarefas**: os veículos a atualizar.
- M — **máquinas**: os setores da rede.
- T — **períodos**: os intervalos do horizonte.

Regras de um escalonamento viável

- Cada tarefa usa **uma máquina por período**, no máximo;
- é **preemptável entre períodos** e fica *ativa* até atender toda a sua demanda;
- C_j é o **último período ativo** de j .

Objetivo original, mono-objetivo: $\min \sum_{j \in J} C_j$ — o tempo total de conclusão.

Minimizar $\sum_j C_j$ não controla o *makespan*

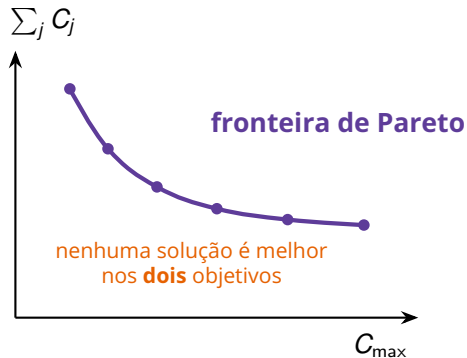
A correlação entre os dois critérios é **fraca**:
um escalonamento ótimo em $\sum_j C_j$ pode
deixar algumas tarefas terminando **muito**
depois das demais.

Propomos o **MOTMDSP**, que minimiza
simultaneamente:

$$\min C_{\max} = \max_{j \in J} C_j \quad (\textit{makespan})$$

$$\min \sum_{j \in J} C_j \quad (\text{tempo total de conclusão})$$

Observado por Andrade et al. (2019).



Notação: conjuntos e parâmetros

Conjuntos

J	Tarefas (veículos a serem atualizados)
M	Máquinas (setores de torres celulares)
T	Períodos-base em um horizonte (indexados por t)

Parâmetros

c_{mt}	Capacidade da máquina m no período t (muu)
η_{mt}	Taxa de produção da máquina m no período t (pu por muu)
q_{jmt}	Consumo de capacidade da tarefa j em m no período t (muu)
d_j	Demanda total da tarefa j (pu)
$r_{\max}$	Limite global de capacidade por período (pu)
$r_{\min}$	Consumo mínimo de recursos por tarefa (pu)
$k_{\max}$	Número máximo de tarefas ativas em um período

Horizonte cíclico: estendemos T para $\tilde{T} = \{0, 1, \dots, t_{\max}\}$ e mapeamos cada $t \in \tilde{T}$ de volta ao período-base $\hat{t} = t \bmod |T|$; daí as versões $c_{m\hat{t}}$, $\eta_{m\hat{t}}$, $q_{j\hat{t}}$ usadas no modelo. muu: *machine utilization units*; pu: *production units*.

Variáveis de decisão

Variável	Significado
$x_{jmt} \in \{0, 1\}$	1 se a tarefa j é atribuída à máquina m no período t
$y_{jmt} \in \mathbb{R}_{\geq 0}$	capacidade da máquina m usada por j no período t (muu)
$s_{jt} \in \{0, 1\}$	1 se a tarefa j inicia no período t
$a_{jt} \in \{0, 1\}$	1 se a tarefa j está ativa em t (escalonada ou em execução)
$f_{jt} \in \{0, 1\}$	1 se a tarefa j termina em t (indicador de <i>pseudo</i> -término)
$C_j \in \mathbb{N}$	tempo de conclusão de j : o índice do período em que j termina
$C_{\max} \in \mathbb{N}$	<i>makespan</i> : o maior tempo de conclusão entre todas as tarefas

Para $t \in \tilde{T} \setminus \{0\}$, $t'(j, t)$ denota o **período predecessor imediato** de t no horizonte cíclico, usado para propagar a atividade da tarefa.

O MOTMDSP é formulado como um programa inteiro misto (MIP) que segue de perto o modelo mono-objetivo de Andrade et al. (2019)

Formulação I: objetivos e lógica de atribuição

$$\min \sum_{j \in J} C_j \quad (1)$$

$$\min C_{\max} \quad (2)$$

$$\text{sujeito a } C_j \geq t a_{jt}, \quad \forall j \in J, \forall t \in \tilde{T} \quad (3)$$

$$\sum_{m \in M} x_{jmt} \leq 1, \quad \forall j \in J, \forall t \in \tilde{T} \quad (4)$$

$$\sum_{t \in \tilde{T}} s_{jt} = 1, \quad \forall j \in J \quad (5)$$

$$a_{jt} \geq \frac{1}{|M|} \sum_{m \in M} x_{jmt}, \quad \forall j \in J, \forall t \in \tilde{T} \quad (6)$$

(3)–(6) definem os tempos de conclusão e a lógica de atribuição e início: C_j é o último período ativo, cada tarefa é atribuída a no máximo uma máquina por período e inicia exatamente uma vez.

Formulação II: atividade, término e consumo

$$a_{j0} = s_{j0}, \quad \forall j \in J \quad (7)$$

$$a_{jt} \leq s_{jt} + a_{jt'}(j,t), \quad \forall j \in J, \forall t \in \tilde{T} \setminus \{0\} \quad (8)$$

$$f_{jt_{\max}} = a_{jt_{\max}}, \quad \forall j \in J \quad (9)$$

$$f_{jt} \leq 1 - \frac{1}{|\tilde{T}|} \sum_{t' > t} a_{jt'}, \quad \forall j \in J, \forall t \in \tilde{T} \setminus \{t_{\max}\} \quad (10)$$

$$y_{jmt} \leq q_{jm\hat{t}} x_{jmt}, \quad \forall j \in J, \forall m \in M, \forall t \in \tilde{T} \quad (11)$$

$$y_{jmt} \geq q_{jm\hat{t}} (x_{jmt} - f_{jt}), \quad \forall j \in J, \forall m \in M, \forall t \in \tilde{T} \quad (12)$$

(7)–(8) propagam a atividade ao longo do horizonte, sem “pular” períodos disponíveis, até a conclusão. (9)–(10) identificam o período final. (11)–(12) conectam y_{jmt} e x_{jmt} : o consumo só é permitido quando a tarefa está atribuída e, exceto no período de término, impõe-se o uso integral $q_{jm\hat{t}}$, evitando subutilização.

Formulação III: capacidades, demanda e *makespan*

$$\sum_{j \in J} y_{jmt} \leq c_{m\hat{t}}, \quad \forall m \in M, \forall t \in \tilde{T} \quad (13)$$

$$\sum_{j \in J} \sum_{m \in M} \eta_{m\hat{t}} y_{jmt} \leq r_{\max}, \quad \forall t \in \tilde{T} \quad (14)$$

$$\sum_{j \in J} a_{jt} \leq k_{\max}, \quad \forall t \in \tilde{T} \quad (15)$$

$$\sum_{m \in M} \sum_{t \in \tilde{T}} \eta_{m\hat{t}} y_{jmt} = d_j, \quad \forall j \in J \quad (16)$$

$$C_{\max} \geq C_j, \quad \forall j \in J \quad (17)$$

(13)–(15) impõem a capacidade de cada máquina, o limite global por período e o máximo de tarefas ativas. (16) garante o atendimento da demanda d_j e (17) define o *makespan* $C_{\max}$. O requisito mínimo $r_{\min}$ é tratado no **pré-processamento**, removendo as tarefas não escalonáveis.

O que vem da literatura, o que é deste trabalho

Da literatura

TMDSP e instâncias reais de FOTA

Andrade et al. (2019)

Decodificador mono-objetivo

Andrade et al. (2019)

NS-BRKGa

Mendes et al. (2022); Pauleti Mendes et al. (2023)

NSGA-II, NSPSO, MOEA/D-DE, MHACO, IHS (PaGMO2)

Deste trabalho

MOTMDSP: extensão biobjetivo. Até onde sabemos, a **primeira** formulação multiobjetivo do problema

Estendido para avaliar os dois objetivos:

$$\mathbf{F} = (C_{\max}, \sum_j C_j)$$

Primeira aplicação ao MOTMDSP, com o motor evolutivo **inalterado**

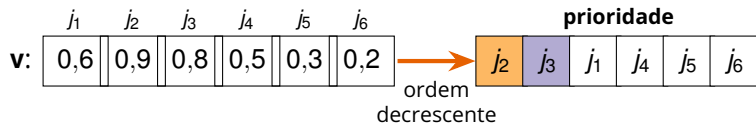
Avaliação em 96 instâncias de larga escala, com o **mesmo decodificador** para todos

Chaves aleatórias definem prioridades de escalonamento

Um cromossomo é um vetor com **uma chave por tarefa**:

$$\mathbf{v} = (v_1, \dots, v_{|J|}), \quad v_j \in [0, 1).$$

Decodificação: ordenar as tarefas por valor **decrescente** da chave $\Rightarrow$ ordem de prioridade.



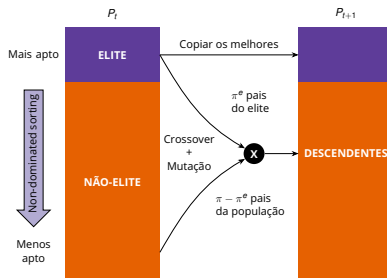
Nessa ordem, cada tarefa ocupa gulosamente os intervalos máquina-período disponíveis mais cedo. C_j é o último período em que a tarefa esteve ativa; alocadas *todas* as tarefas, o decodificador devolve

$$\mathbf{F} = (C_{\max}, \sum_j C_j).$$

Pequenas perturbações nas chaves causam pequenas mudanças na ordem: o espaço de permutações é explorado sem operadores especializados.

Ciclo evolutivo do NS-BRKGA

O motor evolutivo é **independente do problema**: toda a lógica do MOTMDSP está no decodificador.



O elite vem do **non-dominated sorting**, com **crowding distance** e diversidade genotípica no desempate. Compõem ainda o método múltiplas populações com migração, *implicit path-relinking*, *shaking* e *reset* parcial.

O decodificador sempre produz escalonamento viável: sem reparo nem penalidade.

Configuração experimental

Instâncias	96 instâncias reais de FOTA 48 <i>common</i> + 48 <i>rare</i> até 9.706 tarefas, 9.919 máquinas 672 períodos
Algoritmos	NS-BRKGa NSGA-II, NSPSO, MOEA/D-DE, MHACO, IHS (PaGMO2)
Protocolo	5 execuções por algoritmo 900 s por execução Xeon E3-1230 V2, 32 GB, 1 núcleo

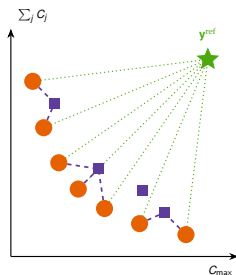
Fronteira de referência

- Uma por instância: todas as soluções não dominadas de **todos** os algoritmos e execuções.
- Filtrada por *non-dominated sorting* + *crowding distance* para até 800 pontos.
- Cada fronteira obtida é podada a até 500 pontos.

Todos os métodos usam o mesmo decodificador: muda apenas o motor de busca.

Como medimos a qualidade de uma fronteira

$NIGD^+$ ($\downarrow$): convergência



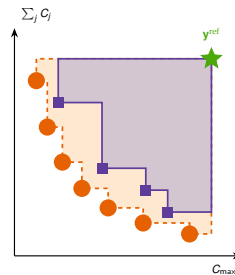
Distância média de cada ponto da referência ao obtido mais próximo, normalizada por y^{ref} .

● fronteira de referência y^{ref}

■ fronteira obtida S

★ pior caso y^{ref}

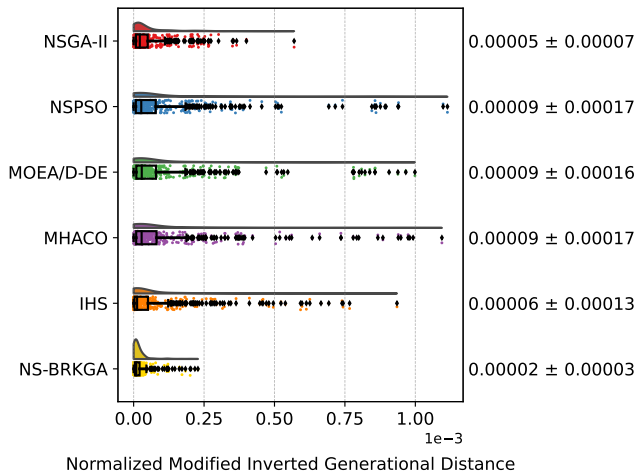
HVR ($\uparrow$): convergência + diversidade



Fração da área dominada pela referência (laranja) que a fronteira obtida (azul) cobre.

$NIGD^+$ perto de 0 é bom; HVR perto de 1 é bom. Ambos ficam em $[0, 1]$.

NS-BRKGA obtém o menor $NIGD^+$ médio global



Média de $NIGD^+$ nas 96 instâncias:

NS-BRKGA 0,00002

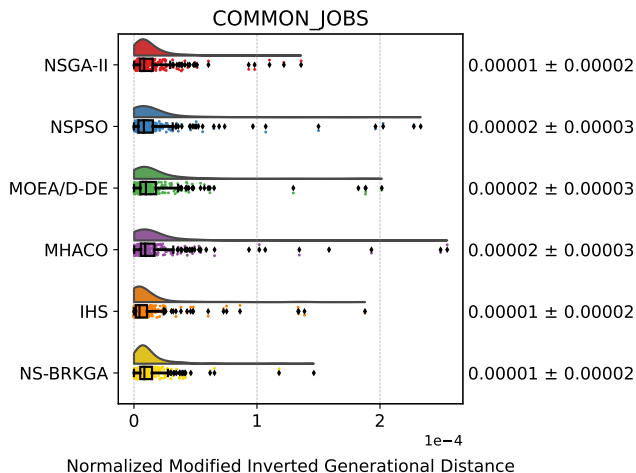
NSGA-II 0,00005

IHS 0,00006

demais 0,00009

Todos superam $HVR \geq 0,9998$, de modo que é o $NIGD^+$ que separa os métodos neste regime. O NS-BRKGA também tem a maior média de HVR e o menor desvio-padrão nos dois indicadores.

Em instâncias `common`, os seis métodos empatam

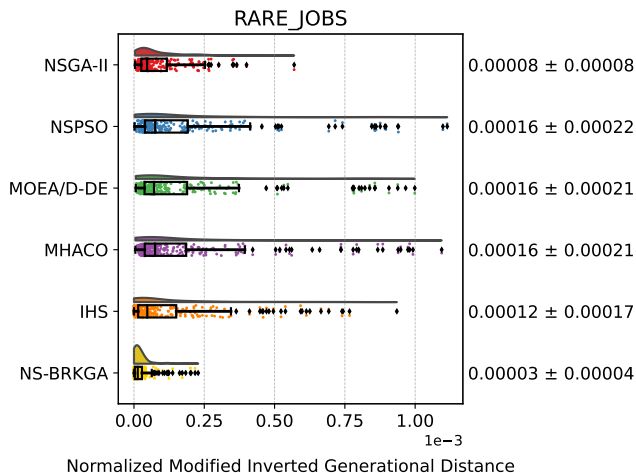


Carros que aparecem em **muitos setores** durante a janela de escalonamento.

Todas as médias de $NIGD^+$ ficam entre **0,00001 e 0,00002**: o NS-BRKA empata com NSGA-II e IHS no melhor valor.

Aqui a escolha do algoritmo **pouco importa**.

A vantagem aparece nas instâncias `rare`



Carros que aparecem em **poucos setores e períodos**. A convergência fica mais difícil para todos.

NS-BRKGa **0,00003**

NSGA-II 0,00008

IHS 0,00012

demais 0,00016

Neste subconjunto o NS-BRKGa também tem o melhor *HVR* médio (0,99997).

Conclusões

- Formulamos o **MOTMDSP**, extensão biobjetivo do TMDSP que trata explicitamente o compromisso entre *makespan* e tempo total de conclusão.
- **Estendemos o decodificador** de Andrade et al. (2019) para os dois objetivos, o que permitiu aplicar ao problema tanto o NS-BRKGA quanto os cinco algoritmos de referência.
- Em 96 instâncias de larga escala, o NS-BRKGA obteve as **melhores médias** de *HVR* e *NIGD*⁺, com vantagem mais nítida nas instâncias *rare*.
- O desempenho **se mantém** à medida que o número de tarefas ou de máquinas cresce, sob o mesmo orçamento de tempo.
- **Uso prático:** em vez de um único escalonamento, o operador recebe um **conjunto de compromissos** entre terminar a campanha mais cedo e concluir mais veículos antes.

Obrigado!

Perguntas?

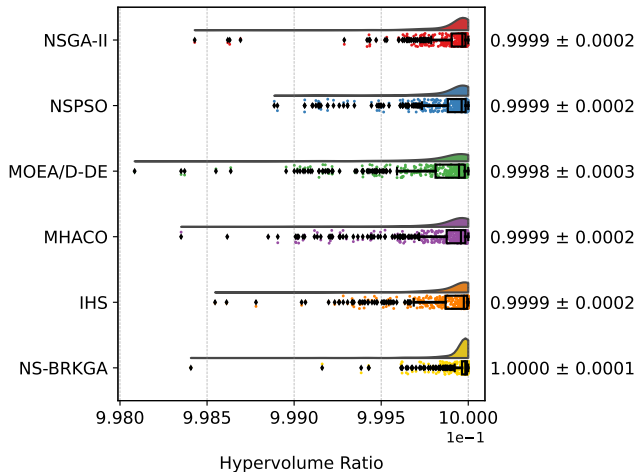
`luis.mendes@ic.unicamp.br`

`github.com/luishpmendes/ns-brkga` · `github.com/ceandrade/tmdsp_instances`

Apoio: parâmetros do NS-BRKGA

Parâmetro	Valor
Tamanho da população $ P $	300
Número de pais π (elitistas π_e)	3 (com $\pi_e = 2$)
Fração de elite $p_{\min}^e - p_{\max}^e$	0,10 – 0,30
Função de viés $\Phi(r)$	$1/\sqrt{r}$
Probabilidade de mutação p_m (η_m)	0,01 ($\eta_m = 50$)
Número de subpopulações	3
Migração	200 gerações, $k = 30$ soluções
<i>Implicit path-relinking</i>	a cada 500 gerações
<i>Shaking</i>	200 gerações sem melhora
<i>Reset parcial</i>	500 gerações sem melhora

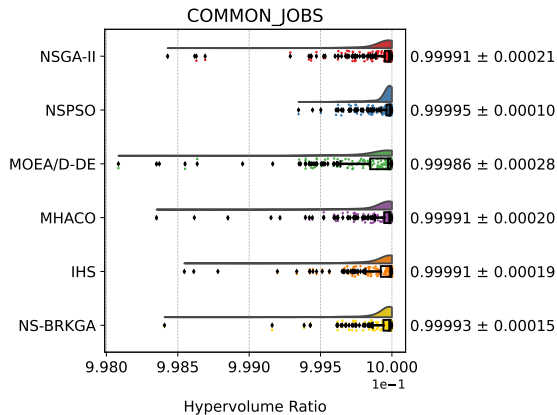
Apoio: *HVR* em todas as instâncias



Todos os algoritmos superam 0,9998 e o NS-BRKGa arredonda para 1,0000 nesta precisão.

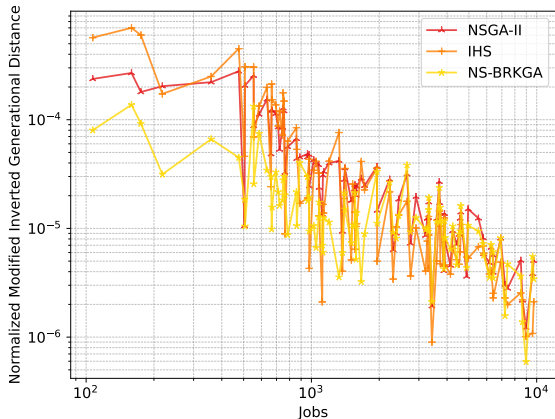
Neste regime o indicador está próximo da saturação, e é o $NIGD^+$ que dá mais resolução entre os métodos.

Apoio: *HVR* por tipo de instância

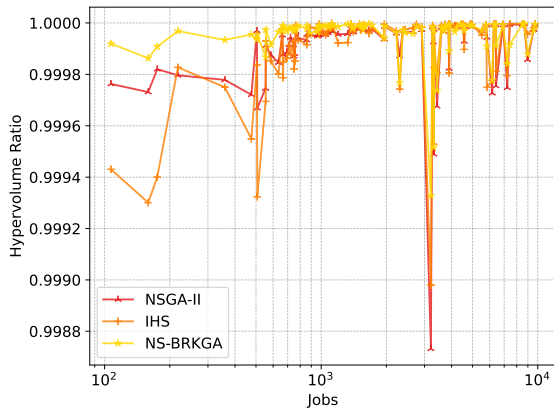


Em `common` o melhor *HVR* médio é do NSPSO (0,99995), com o NS-BRKGA em segundo (0,99993); em `rare` o NS-BRKGA lidera (0,99997).

Apoio: indicadores em função do tamanho da instância



$NIGD^+$ ($\downarrow$) vs. número de tarefas



HVR ($\uparrow$) vs. número de tarefas

Recorte com NSGA-II, IHS e NS-BRKGa, como no artigo. Sem degradação sistemática com o crescimento da instância.